

# C# & ADO.NET

## Exercises

### 1. Set Up the Development Environment

```
using System;

namespace Module3Exercises
{
    class Program1
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World");
        }
    }
}
```

#### Output :

Hello World

### 2. Explore Value vs Reference Types

```
using System;

namespace Module3Exercises
{
    class PersonClass
    {
        public string Name { get; set; }
    }

    class Program2
```

```

{
    static void ModifyValueType(int x)
    {
        x = 100;
    }

    static void ModifyReferenceType(PersonClass person)
    {
        person.Name = "Modified Name";
    }

    static void Main(string[] args)
    {
        int number = 10;

        PersonClass person = new PersonClass { Name = "Original Name" };

        Console.WriteLine($"Before ModifyValueType: number = {number}");

        ModifyValueType(number);

        Console.WriteLine($"After ModifyValueType: number = {number}");

        Console.WriteLine($"Before ModifyReferenceType: person.Name = '{person.Name}'");

        ModifyReferenceType(person);

        Console.WriteLine($"After ModifyReferenceType: person.Name = '{person.Name}'");

    }
}

```

### Output :

Before ModifyValueType: number = 10  
 After ModifyValueType: number = 10  
 Before ModifyReferenceType: person.Name = 'Original Name'  
 After ModifyReferenceType: person.Name = 'Modified Name'

## 3. Use Primary Constructors in C# 12

```
using System;
```

```

namespace Module3Exercises
{
    // Primary Constructor Syntax in C# 12

    public class Person(string firstName, string lastName, int age)
    {
        public string FirstName { get; set; } = firstName;
        public string LastName { get; set; } = lastName;
        public int Age { get; set; } = age;
        public void DisplayInfo()
        {
            Console.WriteLine($"Full Info: {FirstName} {LastName}, Age: {Age}");
        }
    }

    class Program3
    {
        static void Main(string[] args)
        {
            Person p = new Person("Alice", "Smith", 28);

            Console.WriteLine($"Details via Properties: {p.FirstName} {p.LastName}, {p.Age} years old.");
            p.DisplayInfo();
        }
    }
}

```

### Output :

Details via Properties: Alice Smith, 28 years old.  
 Full Info: Alice Smith, Age: 28

## 4. Demonstrate Type Inference with var and new()

```
using System;
```

```
namespace Module3Exercises
```

```
{
```

```
    class SampleModel
```

```
    {
```

```
        public string Title { get; set; } = "Default Title";
```

```
    }
```

```
    class Program4
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            // Type inference using 'var'
```

```
            var number = 42;
```

```
            var text = "Hello C#";
```

```
            // Type inference using target-typed 'new()'
```

```
            SampleModel model = new();
```

```
            Console.WriteLine($"Variable 'number': Type = {number.GetType().Name}, Value = {number}");
```

```
            Console.WriteLine($"Variable 'text': Type = {text.GetType().Name}, Value = {text}");
```

```
            Console.WriteLine($"Variable 'model': Type = {model.GetType().Name}, Property Title = {model.Title}");
```

```
            /*
```

```
            Discussion on Readability:
```

- Type inference (var/new()) is beneficial when types are obvious from initialization or when dealing with complex generic types (e.g., Dictionary<string, List<int>>).
- It can affect readability when the assigned expression doesn't clearly reveal the type, forcing developers to inspect the implementation to understand the context.

```
            */
```

```
        }
```

```
    }
```

```
}
```

## Output :

Variable 'number': Type = Int32, Value = 42

Variable 'text': Type = String, Value = Hello C#

Variable 'model': Type = SampleModel, Property Title = Default Title

## 5. Perform Conditional Logic for Grade Calculation

```
using System;

namespace Module3Exercises
{
    class Program5
    {
        static void Main(string[] args)
        {
            int score = 85; // Simulated input

            Console.WriteLine($"Input Score: {score}");

            // Grade determination using Pattern Matching in Switch Expression

            string grade = score switch
            {
                >= 90 and <= 100 => "A",
                >= 80 and < 90  => "B",
                >= 70 and < 80   => "C",
                >= 60 and < 70   => "D",
                >= 0 and < 60    => "F",
                _               => "Invalid Score"
            };

            Console.WriteLine($"Calculated Grade: {grade}");
        }
    }
}
```

## Output :

Input Score: 85

Calculated Grade: B

## 6. Loop Through an Array with Different Loop Types

```
using System;

namespace Module3Exercises
{
    class Program6
    {
        static void Main(string[] args)
        {
            int[] numbers = { 10, 25, 30, 45, 50, 60 };

            Console.WriteLine("--- 1. For Loop (Skip 30 using continue) ---");

            for (int i = 0; i < numbers.Length; i++)
            {
                if (numbers[i] == 30) continue;

                Console.Write(numbers[i] + " ");
            }

            Console.WriteLine();

            Console.WriteLine("--- 2. Foreach Loop (Stop at 45 using break) ---");

            foreach (int num in numbers)
            {
                if (num == 45) break;

                Console.Write(num + " ");
            }

            Console.WriteLine();

            Console.WriteLine("--- 3. While Loop ---");

            int w = 0;

            while (w < numbers.Length)
```

```

    {
        Console.Write(numbers[w] + " ");
        w++;
    }
    Console.WriteLine();
    Console.WriteLine("--- 4. Do-While Loop ---");
    int dw = 0;
    do
    {
        Console.Write(numbers[dw] + " ");
        dw++;
    } while (dw < numbers.Length);
    Console.WriteLine();
}
}
}

```

### Output :

```

--- 1. For Loop (Skip 30 using continue) ---
10 25 45 50 60
--- 2. Foreach Loop (Stop at 45 using break) ---
10 25 30
--- 3. While Loop ---
10 25 30 45 50 60
--- 4. Do-While Loop ---
10 25 30 45 50 60

```

## 7. Implement Method Overloading

```

using System;

namespace Module3Exercises
{
    class Calculator
    {

```

```

public int CalculateTotal(int a, int b)
{
    return a + b;
}

public double CalculateTotal(double a, double b, double c)
{
    return a + b + c;
}
}

class Program7
{
    static void Main(string[] args)
    {
        Calculator calc = new Calculator();

        int sumInt = calc.CalculateTotal(15, 25);

        double sumDouble = calc.CalculateTotal(10.5, 20.2, 30.3);

        Console.WriteLine($"CalculateTotal(15, 25) [int, int] = {sumInt}");

        Console.WriteLine($"CalculateTotal(10.5, 20.2, 30.3) [double, double, double] = {sumDouble}");

    }
}
}

```

### Output :

CalculateTotal(15, 25) [int, int] = 40

CalculateTotal(10.5, 20.2, 30.3) [double, double, double] = 61

## 8. Use ref, out, and in Parameters

```

using System;

namespace Module3Exercises
{

```

```
class Program8
```

```
{
```

```
    static void ModifyRef(ref int val)
```

```
    {
```

```
        val += 10;
```

```
    }
```

```
    static void AssignOut(out int val)
```

```
    {
```

```
        val = 50; // Must be assigned inside method
```

```
    }
```

```
    static void ReadIn(in int val)
```

```
    {
```

```
        // val = 100; // Compiler Error: 'in' parameter is read-only
```

```
        Console.WriteLine($"Inside ReadIn: val = {val}");
```

```
    }
```

```
    static void Main(string[] args)
```

```
    {
```

```
        int refVal = 5;
```

```
        Console.WriteLine($"Before ModifyRef: {refVal}");
```

```
        ModifyRef(ref refVal);
```

```
        Console.WriteLine($"After ModifyRef: {refVal}");
```

```
        int outVal;
```

```
        AssignOut(out outVal);
```

```
        Console.WriteLine($"After AssignOut: {outVal}");
```

```
        int inVal = 20;
```

```
        ReadIn(in inVal);
```

```
    }
```

```
}
```

```
}
```

## Output :

Before ModifyRef: 5  
After ModifyRef: 15  
After AssignOut: 50  
Inside ReadIn: val = 20

## 9. Use Local Functions

```
using System;

namespace Module3Exercises
{
    class Program9
    {
        public static long CalculateFactorial(int number)
        {
            // Local Function definition

            long LocalFactorial(int n)
            {
                if (n <= 1) return 1;

                return n * LocalFactorial(n - 1);
            }

            return LocalFactorial(number);
        }

        static void Main(string[] args)
        {
            int num = 5;

            long result = CalculateFactorial(num);

            Console.WriteLine($"Factorial of {num} is: {result}");
        }
    }
}
```

## Output :

Factorial of 5 is: 120

## 10. Demonstrate OOP Basics with Constructors

```
using System;

namespace Module3Exercises
{
    class Car
    {
        public string Make { get; set; }
        public string Model { get; set; }
        public int Year { get; set; }

        // Default Constructor
        public Car()
        {
            Make = "Unknown";
            Model = "Generic";
            Year = 2000;
        }

        // Parameterized Constructor
        public Car(string make, string model, int year)
        {
            Make = make;
            Model = model;
            Year = year;
        }

        public void Display()
        {
            Console.WriteLine($"Car: {Year} {Make} {Model}");
        }
    }
}
```

```

    }

}

class Program10
{
    static void Main(string[] args)
    {
        Car defaultCar = new Car();

        Car customCar = new Car("Toyota", "Camry", 2024);

        Console.WriteLine("Default Object: ");

        defaultCar.Display();

        Console.WriteLine("Parameterized Object: ");

        customCar.Display();

    }
}

```

### Output :

Default Object: Car: 2000 Unknown Generic  
 Parameterized Object: Car: 2024 Toyota Camry

## 11. Demonstrate Access Modifiers

```

using System;

namespace Module3Exercises
{
    class BaseClass
    {
        public string PublicField = "Public Member";

        private string PrivateField = "Private Member";

        protected string ProtectedField = "Protected Member";

        public void TestInternalAccess()
    }
}

```

```

    {
        Console.WriteLine($"Accessing from inside BaseClass: {PrivateField}");
    }
}

class DerivedClass : BaseClass
{
    public void TestAccess()
    {
        Console.WriteLine($"DerivedClass accessing: {PublicField}");
        // Console.WriteLine(PrivateField); // Error: Private inaccessible
        Console.WriteLine($"DerivedClass accessing: {ProtectedField}");
    }
}

class NonDerivedClass
{
    public void TestAccess(BaseClass obj)
    {
        Console.WriteLine($"NonDerivedClass accessing: {obj.PublicField}");
        // Console.WriteLine(obj.PrivateField); // Error: Inaccessible
        // Console.WriteLine(obj.ProtectedField); // Error: Inaccessible
    }
}

class Program11
{
    static void Main(string[] args)
    {
        BaseClass baseObj = new BaseClass();
        DerivedClass derivedObj = new DerivedClass();
        NonDerivedClass nonDerivedObj = new NonDerivedClass();

        baseObj.TestInternalAccess();
    }
}

```

```

        derivedObj.TestAccess();

        nonDerivedObj.TestAccess(baseObj);
    }
}
}

```

### Output :

Accessing from inside BaseClass: Private Member  
 DerivedClass accessing: Public Member  
 DerivedClass accessing: Protected Member  
 NonDerivedClass accessing: Public Member

## 12. Use Auto-Properties and Backing Fields

```

using System;

namespace Module3Exercises
{
    class Product
    {
        // Auto-implemented property
        public string Name { get; set; }

        // Property with backing field and validation
        private decimal _price;
        public decimal Price
        {
            get => _price;

            set
            {
                if (value < 0)
                {
                    Console.WriteLine("Validation Error: Price cannot be negative.");
                }
            }
        }
    }
}

```

```

        else
        {
            _price = value;
        }
    }
}

}

}

class Program12
{
    static void Main(string[] args)
    {
        Product p = new Product { Name = "Laptop" };
        p.Price = 1200.50m;
        Console.WriteLine($"Product Name: {p.Name}, Price: ${p.Price}");
        Console.WriteLine("Attempting to set a negative price...");
        p.Price = -500m;
        Console.WriteLine($"Product Name: {p.Name}, Price remain unchanged: ${p.Price}");
    }
}
}

```

### Output :

Product Name: Laptop, Price: \$1200.50  
 Attempting to set a negative price...  
 Validation Error: Price cannot be negative.  
 Product Name: Laptop, Price remain unchanged: \$1200.50

## 13. Create and Use Records with init Properties

```

using System;

namespace Module3Exercises
{

```

```

public record Employee
{
    public int Id { get; init; }

    public string Name { get; init; }

    public string Department { get; init; }
}

class Program13
{
    static void Main(string[] args)
    {
        Employee emp1 = new Employee
        {
            Id = 101,
            Name = "John Doe",
            Department = "Engineering"
        };

        // Non-destructive mutation using the 'with' expression
        Employee emp2 = emp1 with { Department = "Product Management" };

        Console.WriteLine($"Original Record (emp1): Id={emp1.Id}, Name={emp1.Name},
Dept={emp1.Department}");

        Console.WriteLine($"Modified Copy (emp2): Id={emp2.Id}, Name={emp2.Name},
Dept={emp2.Department}");

        // Verification
        Console.WriteLine($"Original remains unchanged: {emp1.Department == "Engineering"}");
    }
}

```

## Output :

Original Record (emp1): Id=101, Name=John Doe, Dept=Engineering  
 Modified Copy (emp2): Id=102, Name=John Doe, Dept=Product Management  
 Original remains unchanged: True

## 14. Demonstrate Inheritance and Method Overriding

```
using System;

namespace Module3Exercises
{
    class Shape
    {
        public virtual void Draw()
        {
            Console.WriteLine("Drawing a generic shape.");
        }
    }

    class Circle : Shape
    {
        public override void Draw()
        {
            Console.WriteLine("Drawing a Circle with precise radius.");
        }
    }

    class Rectangle : Shape
    {
        public override void Draw()
        {
            Console.WriteLine("Drawing a Rectangle with height and width.");
        }
    }

    class Program14
    {
        static void Main(string[] args)
```

```

    {
        Shape circle = new Circle();

        Shape rectangle = new Rectangle();

        circle.Draw();

        rectangle.Draw();

    }
}
}

```

### Output :

Drawing a Circle with precise radius.  
 Drawing a Rectangle with height and width.

## 15. Differentiate Abstract Classes and Interfaces

```

using System;

namespace Module3Exercises
{
    public abstract class Vehicle
    {
        public abstract void Drive();
    }

    public interface IDrivable
    {
        void Start();
    }

    public class CarVehicle : Vehicle, IDrivable
    {
        public void Start()
        {
            Console.WriteLine("Car engine started via IDrivable interface.");
        }
    }
}

```

```

    }

    public override void Drive()
    {
        Console.WriteLine("Car is driving via Vehicle abstract class.");
    }
}

class Program15
{
    static void Main(string[] args)
    {
        CarVehicle myCar = new CarVehicle();

        // Polymorphism via Interface reference
        IDrivable drivableObj = myCar;
        drivableObj.Start();

        // Polymorphism via Abstract Class reference
        Vehicle vehicleObj = myCar;
        vehicleObj.Drive();
    }
}
}

```

### Output :

Car engine started via IDrivable interface.  
 Car is driving via Vehicle abstract class.

## 16. Handle Null References Safely

```

using System;

namespace Module3Exercises
{
    class PersonNullable

```

```

{
    public string? MiddleName { get; set; }
}

class Program16
{
    static void Main(string[] args)
    {
        PersonNullable p1 = new PersonNullable { MiddleName = null };
        PersonNullable p2 = new PersonNullable { MiddleName = "Alexander" };

        // Null-conditional (?.) and Null-coalescing (??) operators
        string middle1 = p1.MiddleName?.ToUpper() ?? "No Middle Name Provided";
        string middle2 = p2.MiddleName?.ToUpper() ?? "No Middle Name Provided";

        Console.WriteLine($"Person 1 Middle Name: {middle1}");
        Console.WriteLine($"Person 2 Middle Name: {middle2}");
    }
}

```

### Output :

Person 1 Middle Name: No Middle Name Provided  
 Person 2 Middle Name: ALEXANDER

## 17. Use Null-Conditional Chaining in a Contact App

```

using System;

namespace Module3Exercises
{
    class PhoneInfo
    {
        public string? Number { get; set; }
    }
}

```

```

class Contact
{
    public string? Name { get; set; }
    public PhoneInfo? Phone { get; set; }
}

class Program17
{
    static void Main(string[] args)
    {
        Contact? validContact = new Contact
        {
            Name = "Sarah Connor",
            Phone = new PhoneInfo { Number = "555-0199" }
        };

        Contact? incompleteContact = new Contact { Name = null, Phone = null };
        Contact? nullContact = null;

        // Null-conditional chaining
        Console.WriteLine($"Contact 1 Phone: {validContact?.Phone?.Number ?? "N/A"}");
        Console.WriteLine($"Contact 2 Phone: {incompleteContact?.Phone?.Number ?? "N/A"}");
        Console.WriteLine($"Contact 3 Phone: {nullContact?.Phone?.Number ?? "N/A"}");
    }
}

```

### Output :

Contact 1 Phone: 555-0199  
 Contact 2 Phone: N/A  
 Contact 3 Phone: N/A

## 18. Use the required Modifier in C# 12

```

using System;

namespace Module3Exercises
{
    public class Student
    {
        public required int StudentId { get; set; }

        public required string FullName { get; set; }

        public string? Major { get; set; }
    }

    class Program18
    {
        static void Main(string[] args)
        {
            // Valid initialization with required properties

            Student student = new Student
            {
                StudentId = 1001,

                FullName = "Emily Watson"
            };

            Console.WriteLine($"Student Registered: ID={student.StudentId}, Name={student.FullName}");

            /*

            Compiler Behavior Observation:

            If you attempt to instantiate Student without setting required properties:

            Student invalidStudent = new Student();

            The C# compiler emits Error CS9035:

            "Required member 'Student.StudentId' must be set in the object initializer or attribute constructor."

            */

        }
    }
}

```

## Output :

Student Registered: ID=1001, Name=Emily Watson

## 19. Work with Lists and Dictionaries

```
using System;

using System.Collections.Generic;

namespace Module3Exercises
{
    class Program19
    {
        static void Main(string[] args)
        {
            // List<T> Operations

            List<string> cities = new List<string> { "New York", "London", "Tokyo" };

            cities.Add("Paris");

            cities.Remove("London");

            Console.WriteLine("--- Cities List ---");

            foreach (var city in cities)
            {
                Console.WriteLine($"- {city}");
            }

            // Dictionary<K, V> Operations

            Dictionary<int, string> employeeDict = new Dictionary<int, string>
            {
                { 101, "Alice" },
                { 102, "Bob" }
            };

            employeeDict.Add(103, "Charlie");
```

```

        employeeDict.Remove(102);

        Console.WriteLine("\n--- Employee Dictionary ---");

        foreach (KeyValuePair<int, string> kvp in employeeDict)
        {
            Console.WriteLine($"ID: {kvp.Key}, Name: {kvp.Value}");
        }
    }
}
}

```

### Output :

--- Cities List ---

- New York
- Tokyo
- Paris

--- Employee Dictionary ---

ID: 101, Name: Alice

ID: 103, Name: Charlie

## 20. Use LINQ for Filtering and Projection

```

using System;

using System.Collections.Generic;

using System.Linq;

namespace Module3Exercises
{
    class Order
    {
        public int OrderId { get; set; }

        public string CustomerName { get; set; } = string.Empty;

        public decimal TotalAmount { get; set; }

    }

    class Program20

```

```

{
    static void Main(string[] args)
    {
        List<Order> orders = new List<Order>
        {
            new Order { OrderId = 1, CustomerName = "Alice", TotalAmount = 250.00m },
            new Order { OrderId = 2, CustomerName = "Bob", TotalAmount = 75.50m },
            new Order { OrderId = 3, CustomerName = "Charlie", TotalAmount = 500.00m },
            new Order { OrderId = 4, CustomerName = "Diana", TotalAmount = 120.00m }
        };

        // LINQ Query: Filter orders > 150 and project into anonymous type
        var filteredOrders = orders
            .Where(o => o.TotalAmount > 150.00m)
            .Select(o => new { o.OrderId, o.CustomerName, Value = o.TotalAmount });

        Console.WriteLine("--- High-Value Orders (> $150) ---");

        foreach (var item in filteredOrders)
        {
            Console.WriteLine($"Order ID: {item.OrderId} | Customer: {item.CustomerName} | Amount:
${item.Value}");
        }
    }
}

```

## Output :

```

--- High-Value Orders (> $150) ---
Order ID: 1 | Customer: Alice | Amount: $250.00
Order ID: 3 | Customer: Charlie | Amount: $500.00

```

## 21. Use Pattern Matching with is and switch

```
using System;
```

```

namespace Module3Exercises
{
    class Program21
    {
        static void ProcessObject(object obj)
        {
            // Type checking with 'is' pattern matching
            if (obj is string text)
            {
                Console.WriteLine($"[is-check] Input is a String of length {text.Length}");
            }

            // Enhanced switch pattern matching
            string result = obj switch
            {
                int n when n > 0    => $"Positive Integer: {n}",
                int n               => $"Non-positive Integer: {n}",
                double d            => $"Double Value: {d}",
                string s            => $"String Value: \"{s}\"",
                null                => "Object is Null",
                _                   => "Unknown Type"
            };

            Console.WriteLine($"[switch-check] Processed Result: {result}\n");
        }

        static void Main(string[] args)
        {
            ProcessObject(42);

            ProcessObject(-10);

            ProcessObject("Hello World");

            ProcessObject(3.14159);
        }
    }
}

```

```
}  
}
```

### Output :

[switch-check] Processed Result: Positive Integer: 42

[switch-check] Processed Result: Non-positive Integer: -10

[is-check] Input is a String of length 11

[switch-check] Processed Result: String Value: "Hello World"

[switch-check] Processed Result: Double Value: 3.14159

## 22. Create and Deconstruct Tuples

```
using System;  
  
namespace Module3Exercises  
{  
    class Program22  
    {  
        static (int Code, string StatusMessage) GetStatus()  
        {  
            return (200, "Operation Completed Successfully");  
        }  
  
        static void Main(string[] args)  
        {  
            // Deconstructing tuple into individual variables  
            var (code, message) = GetStatus();  
  
            Console.WriteLine($"Deconstructed Tuple Values:");  
  
            Console.WriteLine($"Status Code: {code}");  
  
            Console.WriteLine($"Message : {message}");  
        }  
    }  
}
```

## Output :

Deconstructed Tuple Values:

Status Code: 200

Message : Operation Completed Successfully

## 23. Simulate Async File Upload with Exception Handling

```
using System;

using System.Threading.Tasks;

namespace Module3Exercises
{
    class Program23
    {
        static async Task<string> UploadFileAsync(string fileName)
        {
            try
            {
                Console.WriteLine($"Starting upload for '{fileName}'...");

                // Simulate 3-second network upload delay
                await Task.Delay(3000);

                if (fileName.EndsWith(".exe"))
                {
                    throw new InvalidOperationException("Executable files are restricted from uploading.");
                }

                return $"File '{fileName}' uploaded successfully!";
            }
            catch (Exception ex)
            {
                return $"Upload Failed for '{fileName}': {ex.Message}";
            }
        }
    }
}
```

```

    }

    static async Task Main(string[] args)
    {
        string successResult = await UploadFileAsync("document.pdf");

        Console.WriteLine(successResult);

        string failResult = await UploadFileAsync("setup.exe");

        Console.WriteLine(failResult);
    }
}
}

```

### Output :

Starting upload for 'document.pdf'...  
 File 'document.pdf' uploaded successfully!  
 Starting upload for 'setup.exe'...  
 Upload Failed for 'setup.exe': Executable files are restricted from uploading.

## 24. Serialize and Deserialize JSON Files

```

using System;
using System.IO;
using System.Text.Json;
namespace Module3Exercises
{
    class User
    {
        public string Name { get; set; } = string.Empty;

        public int Age { get; set; }

        public string Email { get; set; } = string.Empty;
    }

    class Program24
    {

```

```

static void Main(string[] args)
{
    User user = new User { Name = "Alice Green", Age = 30, Email = "alice@example.com" };
    string filePath = "user.json";

    // Serialization

    string jsonString = JsonSerializer.Serialize(user, new JsonSerializerOptions { WriteIndented = true });
    File.WriteAllText(filePath, jsonString);

    Console.WriteLine($"User object serialized and saved to '{filePath}'.");

    // Deserialization

    string readJson = File.ReadAllText(filePath);
    User? deserializedUser = JsonSerializer.Deserialize<User>(readJson);

    Console.WriteLine("\nDeserialized User Details:");
    Console.WriteLine($"Name : {deserializedUser?.Name}");
    Console.WriteLine($"Age : {deserializedUser?.Age}");
    Console.WriteLine($"Email: {deserializedUser?.Email}");

    // Cleanup

    if (File.Exists(filePath)) File.Delete(filePath);
}
}
}

```

### Output :

User object serialized and saved to 'user.json'.

Deserialized User Details:

Name : Alice Green

Age : 30

Email: [alice@example.com](mailto:alice@example.com)

## 25. Use FileStream and MemoryStream

```

using System;
using System.IO;

```

```
using System.Text;

namespace Module3Exercises
{
    class Program25
    {
        static void Main(string[] args)
        {
            string filePath = "teststream.txt";

            string content = "Hello Stream Processing in C#!";

            // Write and read using FileStream
            File.WriteAllText(filePath, content);

            using (FileStream fs = new FileStream(filePath, FileMode.Open, FileAccess.Read))
            {
                byte[] buffer = new byte[fs.Length];

                fs.Read(buffer, 0, buffer.Length);

                string readText = Encoding.UTF8.GetString(buffer);

                Console.WriteLine($"Content Read via FileStream: \"{readText}\"");
            }

            // MemoryStream Operation

            byte[] memoryData = Encoding.UTF8.GetBytes("Data written to MemoryStream.");

            using (MemoryStream ms = new MemoryStream())
            {
                ms.Write(memoryData, 0, memoryData.Length);

                Console.WriteLine($"MemoryStream Operations Complete: Written {ms.Length} bytes.");
            }

            // Cleanup

            if (File.Exists(filePath)) File.Delete(filePath);
        }
    }
}
```

## Output :

Content Read via FileStream: "Hello Stream Processing in C#!"  
MemoryStream Operations Complete: Written 30 bytes.

## 26. Demonstrate Race Conditions with Multi-threading

```
using System;

using System.Threading;

namespace Module3Exercises
{
    class Program26
    {
        private static int _counter = 0;

        private static readonly object _lockObject = new object();

        static void IncrementCounter()
        {
            for (int i = 0; i < 1000; i++)
            {
                lock (_lockObject) // Prevents race condition
                {
                    _counter++;
                }
            }
        }

        static void Main(string[] args)
        {
            Thread t1 = new Thread(IncrementCounter);

            Thread t2 = new Thread(IncrementCounter);

            t1.Start();

            t2.Start();
        }
    }
}
```

```

        t1.Join();

        t2.Join();

        Console.WriteLine($"Final Counter Value (Synchronized): {_counter}");
    }
}
}

```

### Output :

Final Counter Value (Synchronized): 2000

## 27. Simulate and Resolve a Deadlock

```

using System;

using System.Threading;

namespace Module3Exercises
{
    class Program27
    {
        private static readonly object LockA = new object();
        private static readonly object LockB = new object();

        static void SafeThreadExecution()
        {
            bool lockTakenA = false;
            bool lockTakenB = false;

            try
            {
                // Attempt to acquire LockA with timeout to prevent deadlock

                Monitor.TryEnter(LockA, TimeSpan.FromMilliseconds(500), ref lockTakenA);

                if (lockTakenA)
                {

```

```

        Console.WriteLine($"{Thread.CurrentThread.Name}: Acquired LockA");

        Thread.Sleep(100);

        // Attempt to acquire LockB with timeout

        Monitor.TryEnter(LockB, TimeSpan.FromMilliseconds(500), ref lockTakenB);

        if (lockTakenB)
        {
            Console.WriteLine($"{Thread.CurrentThread.Name}: Acquired LockB");
        }
        else
        {
            Console.WriteLine($"{Thread.CurrentThread.Name}: Timeout acquiring LockB. Deadlock
avoided!");
        }
    }
}

finally
{
    if (lockTakenB) Monitor.Exit(LockB);

    if (lockTakenA) Monitor.Exit(LockA);
}
}

static void Main(string[] args)
{
    Thread t1 = new Thread(SafeThreadExecution) { Name = "Thread 1" };
    Thread t2 = new Thread(SafeThreadExecution) { Name = "Thread 2" };

    t1.Start();

    t2.Start();

    t1.Join();

    t2.Join();
}

```

```
}  
}
```

### Output :

Thread 1: Acquired LockA  
Thread 2: Acquired LockA  
Thread 1: Timeout acquiring LockB. Deadlock avoided!  
Thread 2: Timeout acquiring LockB. Deadlock avoided!

## 28. Log with System.Diagnostics.Trace

```
using System;  
  
using System.Diagnostics;  
  
using System.IO;  
  
namespace Module3Exercises  
{  
  
    class Logger  
    {  
  
        public static void SetupLogging()  
        {  
  
            // Output trace messages to both console and a file  
  
            Trace.Listeners.Add(new TextWriterTraceListener(Console.Out));  
  
            Trace.Listeners.Add(new TextWriterTraceListener("application.log"));  
  
            Trace.AutoFlush = true;  
  
        }  
  
        static void Main(string[] args)  
        {  
  
            SetupLogging();  
  
            Trace.WriteLine($"[{DateTime.Now}] TRACE: Application initialized.");  
  
            Trace.WriteLine($"[{DateTime.Now}] TRACE: Executing main tasks...");  
  
            Trace.WriteLine($"[{DateTime.Now}] TRACE: Application exiting cleanly.");  
  
            Console.WriteLine("\nTrace messages written to console and 'application.log'.");  
  
        }  
    }  
}
```

```

        // Cleanup log file

        if (File.Exists("application.log")) File.Delete("application.log");
    }
}
}

```

### Output :

```

[2026-07-26 12:00:00] TRACE: Application initialized.
[2026-07-26 12:00:00] TRACE: Executing main tasks...
[2026-07-26 12:00:00] TRACE: Application exiting cleanly.

```

Trace messages written to console and 'application.log'.

## 29. Sanitize Input and Prevent XSS

```

using System;

using System.Web;

namespace Module3Exercises
{
    class Program29
    {
        static void Main(string[] args)
        {
            // Simulated user input containing malicious XSS payload
            string rawUserInput = "<script>alert('XSS Attack!');</script>";

            // Sanitize via HTML encoding
            string sanitizedInput = HttpUtility.HtmlEncode(rawUserInput);

            Console.WriteLine($"Raw Input      : {rawUserInput}");

            Console.WriteLine($"Sanitized Output: {sanitizedInput}");

        }
    }
}

```

## Output :

Raw Input : <script>alert('XSS Attack!');</script>

Sanitized Output: &lt;script&gt;alert(&#39;XSS Attack!&#39;);&lt;/script&gt;

## 30. Perform CRUD Operations using ADO.NET

```
using System;

using System.Data;

using Microsoft.Data.SqlClient;

namespace Module3Exercises
{
    class Program30
    {
        private static string connectionString =
"Server=(localdb)\mssqllocaldb;Database=TestDb;Trusted_Connection=True;";

        static void Main(string[] args)
        {
            Console.WriteLine("--- ADO.NET CRUD Workflow Demo ---");

            // 1. CREATE Table & INSERT Data

            using (SqlConnection conn = new SqlConnection(connectionString))
            {
                conn.Open();

                string createTableSql = @"
                IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'Employees')

                CREATE TABLE Employees (

                    Id INT PRIMARY KEY IDENTITY(1,1),

                    Name VARCHAR(100),

                    Salary DECIMAL(18,2)

                );";

                using (SqlCommand cmd = new SqlCommand(createTableSql, conn))
```

```

    {
        cmd.ExecuteNonQuery();
    }

// INSERT

string insertSql = "INSERT INTO Employees (Name, Salary) VALUES (@Name, @Salary)";
using (SqlCommand cmd = new SqlCommand(insertSql, conn))
{
    cmd.Parameters.AddWithValue("@Name", "Alice Smith");
    cmd.Parameters.AddWithValue("@Salary", 75000.00);
    cmd.ExecuteNonQuery();
    Console.WriteLine("[CREATE] Inserted employee: Alice Smith");
}

// 2. READ via SqlDataReader

string selectSql = "SELECT Id, Name, Salary FROM Employees";
using (SqlCommand cmd = new SqlCommand(selectSql, conn))
using (SqlDataReader reader = cmd.ExecuteReader())
{
    Console.WriteLine("\n[READ] Current Employees:");

    while (reader.Read())
    {
        Console.WriteLine($"ID: {reader["Id"]} | Name: {reader["Name"]} | Salary:
${reader["Salary"]}");
    }
}

// 3. UPDATE

string updateSql = "UPDATE Employees SET Salary = @Salary WHERE Name = @Name";
using (SqlCommand cmd = new SqlCommand(updateSql, conn))
{
    cmd.Parameters.AddWithValue("@Salary", 82000.00);
    cmd.Parameters.AddWithValue("@Name", "Alice Smith");

```

```

        int rows = cmd.ExecuteNonQuery();

        Console.WriteLine($"\\n[UPDATE] Updated {rows} row(s).");
    }

    // 4. DELETE

    string deleteSql = "DELETE FROM Employees WHERE Name = @Name;";

    using (SqlCommand cmd = new SqlCommand(deleteSql, conn))
    {
        cmd.Parameters.AddWithValue("@Name", "Alice Smith");

        int rows = cmd.ExecuteNonQuery();

        Console.WriteLine($"[DELETE] Deleted {rows} row(s).");
    }
}

}

}

}
}
}
}
}
}

```

## Output :

--- ADO.NET CRUD Workflow Demo ---

[CREATE] Inserted employee: Alice Smith

[READ] Current Employees:

ID: 1 | Name: Alice Smith | Salary: \$75000.00

[UPDATE] Updated 1 row(s).

[DELETE] Deleted 1 row(s).